

УЧЕБНИК ПОЛЬЗОВАТЕЛЯ

Compliance Profile Builder

Конструктор профилей соответствия XCCDF/OVAL
для Kaspersky Vulnerability Management

Содержание

Для чего нужен compliance-профиль	4
Два документа внутри архива	4
Почему шагов десять.....	4
Что нужно знать заранее.....	4
Основная идея – цепочка зависимостей.....	5
Что можно переиспользовать.....	5
Шаг 1. Бенчмарк — титульный лист документа	6
Поля.....	6
Про статусы.....	6
Как именовать	6
Шаг 2. OVAL-файл — контейнер для технической части	7
Когда файлов нужно несколько.....	7
Шаг 3. Переменные — эталонные значения в одном месте.....	8
Когда переменная окупается	8
Поля.....	8
Три примера	8
Шаг 4. Объекты — где искать.....	9
Доступные типы проверок.....	9
Пример 1. Строка в конфигурационном файле	9
Пример 2. Настройка ядра.....	10
Пример 3. Параметр реестра Windows	10
Пример 4. Объект без полей.....	10
Совет.....	10
Шаг 5. Состояния — с чем сравнить	11
Когда состояние не нужно	11
Операции сравнения.....	11
Типы данных — главная ловушка шага.....	11
Два способа задать значение.....	11
Пример 1. Число из текстового файла	12
Пример 2. Число от системы	12
Пример 3. Значение напрямую	12
Пример 4. Два поля сразу.....	12
Шаг 6. Тесты — соединяем объект и состояние.....	13
Поля.....	13
Два условия, которые путают	13
Пример 1. Обычная проверка значения.....	14
Пример 2. Проверка наличия — без состояния	14
Пример 3. Проверка от обратного	14
Шаг 7. Определения — законченная проверка	15

Поля.....	15
Семейство ОС и платформы — это разное.....	15
Критерии: один тест или несколько	15
Что конструктор делает сам	16
Пример.....	16
Шаг 8. Правила — требование в документе	17
Поля.....	17
Идентификатор требования — самое важное поле шага	17
Галочка «Выбрано по умолчанию»	17
Пример.....	18
Чем правило отличается от определения.....	18
Шаг 9. Профили — набор правил для запуска.....	19
Поля.....	19
Зачем несколько профилей.....	19
Изменение состава позже	19
Пример.....	19
Шаг 10. Проверка и экспорт	20
Что проверяется	20
Ошибки и замечания	20
Структура архива.....	20
Повторный экспорт и доработка	20
Три сквозных примера	21
Пример 1. Пакет аудита установлен	21
Пример 2. Переадресация пакетов отключена	21
Пример 3. Минимальная длина пароля.....	21
Сравнение трёх подходов.....	22
Механики интерфейса.....	23
Мастер и конструктор	23
Язык и тема	23
Редактирование и удаление.....	23
Импорт готового архива	23
Хранение данных.....	23
Частые ошибки.....	24
Валидатор нашёл проблему.....	24
В списке нечего выбрать	24
Профиль загрузился, но результат неожиданный	24
Как проверить себя.....	24
Словарь терминов	25
Два формата.....	25
Структура идентификатора OVAL.....	26

Для чего нужен compliance-профиль

Он описывает, как должен быть настроен защищённый актив.

Результатом работы данного сервиса является ZIP-архив, который Kaspersky Vulnerability Management загружает и применяет для проверки активов.

Пример требования: «минимальная длина пароля — не меньше 14 символов». Человеку этого достаточно. Программному продукту нужно объяснить всё по буквам: в каком файле искать, какую строку, какое число из неё взять, с чем сравнить и что считать нарушением. Конструктор — это форма, которая превращает первую формулировку во вторую.

Два документа внутри архива

В архиве всегда два файла, и они делят работу:

Файл	Что описывает	Пример содержимого
benchmark-xccdf.xml	Требования человеческим языком: что проверяем и зачем	«Минимальная длина пароля», описание, ссылка на пункт стандарта
checks-oval.xml	Техническую логику: где искать и с чем сравнивать	Файл /etc/security/pwquality.conf, параметр minlen, значение ≥ 14

XCCDF отвечает на вопрос «что требуется», OVAL — «как это проверить». Правило из первого файла ссылается на определение из второго. Разорвана ссылка — архив не примут.

Почему шагов десять

Проверка собирается не целиком, а из мелких деталей: отдельно «где искать», отдельно «с чем сравнить», отдельно «как это назвать в отчёте». Каждый шаг мастера добавляет одну такую деталь. Это кажется избыточным для одной проверки, но окупается на десятой: детали переиспользуются, а не копируются.

Что нужно знать заранее

Ничего. Идентификаторы конструктор подставляет сам, семейство ОС определяет по типу проверки, формат соблюдает автоматически. Знание XCCDF и OVAL пригодится, но не обязательно — учебник объясняет каждое понятие там, где оно впервые встречается.

Основная идея – цепочка зависимостей

Каждый элемент профиля ссылается на предыдущий и не может существовать без него. Отсюда и порядок шагов.



Рисунок 1. Цепочка зависимостей элементов профиля

Читается так: объект говорит, **где искать**, состояние — **с чем сравнить**, тест соединяет их в одну проверку. Определение даёт проверке имя и описание, правило превращает её в требование, профиль собирает требования в набор для запуска.

Если попытаться начать с конца — создать профиль первым, — выбирать будет не из чего: правил ещё нет. Поэтому мастер ведёт снизу вверх, а если элемента предыдущего шага не хватает, вместо пустого списка показывает плашку с кнопкой перехода.

Что можно переиспользовать

Связи не обязаны быть «один к одному» — на этом экономится много работы:

Элемент	Сколько раз используется
Переменная	одна на несколько состояний: например, «14» для длины пароля в разных проверках
Объект	один на несколько тестов: файл <code>sshd_config</code> проверяется на разные параметры
Определение	одно на несколько правил в разных бенчмарках
Правило	входит в несколько профилей: «Базовый» и «Усиленный»

Шаг 1. Бенчмарк — титульный лист документа

Бенчмарк — это весь документ с требованиями целиком. На этом шаге ему даётся название и описание — то, что увидит администратор в списке загруженных профилей.

Аналогия: если профиль — это книга требований, то бенчмарк — её обложка и выходные данные. Сами требования появятся позже.

Поля

Поле	Что писать	Пример
Идентификатор проекта	Имя папки на сервере. Только латиница, цифры, - и _	profile_astra
Идентификатор benchmark	Имя документа внутри XML	xccdf_custom_benchmark_astra
Язык (xml:lang)	Язык текстов в документе	ru-RU
Статус	Готовность документа	Принят
Заголовок	Название	Политика безопасности Astra Linux
Описание	Зачем этот набор, и к чему применяется	Базовые требования к настройке серверов подразделения
Версия	Версия вашего документа	1.0

Про статусы

Статус ничего не меняет в работе проверок — это пометка для людей:

- **Черновик** — ещё собирается, не согласован
- **Промежуточный** — на согласовании
- **Принят** — утверждён, используется в работе. Обычный выбор
- **Устарел** — заменён другим набором

Как именовать

Идентификаторы видны в отчётах и журналах, поэтому полезно договориться о единой схеме заранее. Рабочий вариант — `xccdf_<организация>_benchmark_<система>`, например `xccdf_acme_benchmark_astra`. Менять идентификатор позже нежелательно: для системы это будет другой документ, а не новая версия старого.

Заголовок и описание можно править в любой момент — вернитесь на этот шаг и нажмите «Сохранить».

Шаг 2. OVAL-файл — контейнер для технической части

На этом шаге создаётся второй файл архива — тот, где будет лежать вся механика проверок. Шаг занимает полминуты: достаточно задать имя файла.

Поле	Значение по умолчанию	Менять?
Имя файла	checks-oval.xml	Обычно нет
Название продукта	Custom	Нет
Версия продукта	1.0	Нет
Версия схемы OVAL	5.11.2	Нет

Имя файла должно заканчиваться на `-oval.xml` — это требование формата архива.

Когда файлов нужно несколько

Для обычного профиля достаточно одного. Несколько имеют смысл в двух случаях:

1. **Проверок очень много** — сотни. Разделить по темам проще, чем искать нужную в одном длинном списке: `checks-network-oval.xml`, `checks-accounts-oval.xml`
2. **Разные системы в одном бенчмарке** — проверки Linux и Windows лежат отдельно

Разбиение ничего не ускоряет и не меняет результат — это только вопрос порядка. Важное ограничение: **ссылаться между файлами нельзя**. Тест из одного файла не увидит объект из другого, поэтому вся цепочка одной проверки живёт внутри одного файла.

Если файлов несколько, над списком элементов появляются вкладки с именами файлов — переключайтесь между ними перед созданием элемента.

Шаг 3. Переменные — эталонные значения в одном месте

Переменная хранит число или строку, с которой потом сравнивается реальная настройка узла. Главный смысл — задать требование один раз и ссылаться на него отовсюду.

Этот шаг можно пропустить — значение разрешается ввести прямо в состоянии на шаге 5.

Когда переменная окупается

Представьте: в профиле шесть проверок ссылаются на длину пароля 14 символов. Служба информационной безопасности поднимает требование до 16.

- **С переменной:** правите одно значение
- **Без переменной:** открываете шесть состояний и ни одно не забываете

Второй плюс — читаемость: в списке переменных сразу видны все числовые требования профиля.

Поля

Поле	Пояснение
Идентификатор	Подставляется сам: <code>oval:custom:var:1</code>
Тип данных	Что за значение: целое число, строка, логическое, версия
Значение	Сам эталон: <code>14, 0, ^no\$</code>
Комментарий	Зачем она нужна — помогает через время

Три примера

1. Числовое требование. История паролей — не менее 24 записей.

- Тип данных: Целое число, значение: 24
- Комментарий: Минимальная глубина истории паролей

2. Точное слово. Параметр `PermitRootLogin` должен быть `no`.

- Тип данных: Строка, значение: `^no$`
- Знаки `^` и `$` означают «вся строка целиком». Без них подошло бы и `nowhere`

3. Диапазон значений. Длина пароля — 14 и больше, но читается она из текстового файла как строка.

- Тип данных: Строка, значение: `^(1[4-9]|[2-9][0-9]|[0-9]{3,})$`
- Расшифровка: `1[4-9]` — от 14 до 19, `[2-9][0-9]` — от 20 до 99, `[0-9]{3,}` — 100 и больше

Третий пример показывает важную тонкость: если значение достаётся из текстового файла, оно приходит строкой, и сравнивать его приходится по шаблону, а не через «больше или равно». Где система отдаёт число сразу — например, парольная политика Windows — берётся целое число и обычное сравнение.

Шаг 4. Объекты — где искать

Объект отвечает на один вопрос: какой кусочек системы нужно посмотреть. Не «правильно ли он настроен» — это будет на шаге 5, а именно «где смотреть».

Первым делом выбирается **тип проверки** — от него зависят остальные поля формы. Выбрали «Файл» — появятся путь и имя файла; выбрали «Параметр реестра» — куст, ключ и имя параметра.

Доступные типы проверок

Их восемнадцать — ровно те, что разрешены форматом. Остальные типы из стандарта OVAL не поддерживаются.

Тип проверки	Что смотрит	Типичное применение
Содержимое текстового файла	Строку в файле по шаблону	Параметры в <code>sshd_config</code> , <code>pwquality.conf</code>
Файл (права, владелец)	Атрибуты файла	Владелец и права на <code>/etc/shadow</code>
Параметр <code>sysctl</code>	Настройку ядра	Сетевые параметры, ASLR
Учётная запись (<code>/etc/passwd</code>)	Сведения о пользователе	Заблокирован ли вход
Пакет DEB (<code>dpkg</code>)	Установленный пакет	Astra Linux, Debian, Ubuntu
Пакет RPM	Установленный пакет	РЕД ОС, ALT Linux
Точка монтирования	Параметры монтирования	<code>noexec</code> на <code>/tmp</code>
Прослушиваемые порты	Открытые сетевые порты	Запрет telnet, FTP
Свойство <code>systemd</code> -юнита	Состояние службы	Активен ли <code>auditd</code>
Значение переменной	Значение другой переменной OVAL	Составные проверки
Параметр реестра	Значение в реестре Windows	Отключение SMBv1
Политика паролей	Парольную политику Windows	Длина, история, сложность
Политика блокировки	Блокировку учётных записей	Порог неудачных попыток
Политика аудита	Подкатегории аудита	Аудит входа в систему
Право пользователя	Назначенные права	Кто может входить как служба
SID учётной записи	Идентификатор безопасности	Сопоставление учётных записей
WMI-запрос	Любые сведения через WMI	То, чего нет в других типах

Пример 1. Строка в конфигурационном файле

Нужно проверить `minlen` в `/etc/security/pwquality.conf`.

Поле	Значение
Тип проверки	Содержимое текстового файла
Путь к каталогу	<code>/etc/security</code>
Имя файла	<code>pwquality.conf</code>
Регулярное выражение	<code>^[[:space:]]*minlen[[:space:]]*=[[:space:]]*([0-9]+)</code>

Разбор выражения по частям:

- `^` — начало строки
- `[[:space:]]*` — любое количество пробелов, включая ноль
- `minlen` — само имя параметра
- `([0-9]+)` — **захватывающая группа**: число в скобках будет извлечено и передано состоянию

Скобки — самая важная деталь. Без них состоянию достанется вся строка `minlen = 14`, а сравнивать нужно только 14.

Пример 2. Настройка ядра

Проверяем, что переадресация пакетов выключена.

Поле	Значение
Тип проверки	Параметр sysctl
Имя параметра	<code>net.ipv4.ip_forward</code>

Здесь поле всего одно: система умеет спрашивать значение параметра напрямую, без разбора файлов.

Пример 3. Параметр реестра Windows

Отключён ли устаревший протокол SMBv1.

Поле	Значение
Тип проверки	Параметр реестра
Куст (hive)	<code>HKEY_LOCAL_MACHINE</code>
Ключ (key)	<code>SYSTEM\CurrentControlSet\Services\mrxsmb10</code>
Имя параметра	<code>Start</code>

Пример 4. Объект без полей

У парольной политики Windows и политики блокировки полей нет вовсе — форма покажет только идентификатор и комментарий. Это нормально: политика в системе одна, уточнять нечего, и вся проверка ложится на состояние.

Совет

Комментарий у объекта не обязателен, но заполнять его стоит: именно он показывается в выпадающих списках на шаге с тестами. Сравните: `oval:custom:obj:3` против `oval:custom:obj:3` — Параметр ядра `net.ipv4.ip_forward`.

Шаг 5. Состояния — с чем сравнить

Состояние описывает правильное значение: чему должна равняться настройка, больше или меньше чего быть. Объект говорит «где смотреть», состояние — «что там должно быть».

Тип проверки выбирается **тот же**, что и у объекта: состояние для файла нельзя применить к параметру реестра.

Когда состояние не нужно

Если проверяется сам факт наличия — пакет установлен, порт не открыт — состояние не создаётся. Всю работу сделает поле «Условие существования» на шаге 6.

Операции сравнения

Операция	Когда применять	Пример
Равно	Точное совпадение	<code>ip_forward = 0</code>
Не равно	Любое, кроме указанного	Оболочка не <code>/bin/bash</code>
Больше или равно	Нижняя граница	Длина пароля ≥ 14
Меньше или равно	Верхняя граница	Порог блокировки ≤ 5
Больше / Меньше	Строгая граница	Срок действия > 0
Соответствует регулярному выражению	Текст по шаблону	Значение из текстового файла

Типы данных — главная ловушка шага

Тип данных решает, **как** сравниваются значения. Сравнение строк идёт по алфавиту, а не по величине: «9» > «14», потому что символ 9 идёт после 1. Для чисел всегда ставьте Целое число.

Отдельно запомните: у параметра `sysctl` тип по умолчанию — строка. Если сравниваете числа, переключите на Целое число вручную.

Тип	Для чего
Целое число	Любые числовые сравнения
Строка	Текст, пути, регулярные выражения
Логическое	Включено / выключено: <code>true</code> , <code>false</code>
Версия	Сравнение версий: 2.4.10 выше 2.4.9

Два способа задать значение

Рядом с полем есть галочка **«Из переменной»**:

3. **Галочка снята** — значение вводится прямо в поле
4. **Галочка поставлена** — поле ввода сменяется списком переменных с шага 3

Важно: списки **«операция»** и **«тип данных»** остаются видимы в **обоих режимах** и заполнять их нужно всегда. Переменная задаёт только само значение, а не способ сравнения.

Пример 1. Число из текстового файла

Продолжаем пример с `minlen`. Объект уже извлёк число скобками.

Поле	Значение
Тип проверки	Содержимое текстового файла
Поле <code>subexpression</code>	Из переменной → <code>oval:custom:var:1</code>
Операция	Соответствует регулярному выражению
Тип данных	Строка

Почему строка, а не число: значение пришло из текстового файла, и проверяется оно шаблоном «14 и больше» из переменной.

Пример 2. Число от системы

Параметр ядра должен равняться нулю.

Поле	Значение
Тип проверки	Параметр <code>sysctl</code>
Поле <code>value</code>	Из переменной → <code>oval:custom:var:3</code>
Операция	Равно
Тип данных	Целое число — переключить вручную

Пример 3. Значение напрямую

Владелец файла `/etc/passwd` — `root`. Переменная тут избыточна: ноль не изменится никогда.

Поле	Значение
Тип проверки	Файл (права, владелец)
Поле <code>uid</code>	0 — галочка снята
Операция	Равно
Тип данных	Целое число

Пример 4. Два поля сразу

Аудит входа в систему ведётся и для успешных, и для неудачных попыток.

Поле	Значение	Операция	Тип
<code>audit_success</code>	<code>true</code>	Равно	Логическое
<code>audit_failure</code>	<code>true</code>	Равно	Логическое

Заполненные поля объединяются по «И»: верным считается состояние, где совпали все заполненные условия. Пустые поля не проверяются — заполняйте только нужное.

Шаг 6. Тесты — соединяем объект и состояние

Тест говорит: «возьми этот объект и сравни его с этим состоянием». Это самый простой шаг: всё содержание уже создано, осталось выбрать из списков.

Списки отфильтрованы по типу проверки: выбрали Параметр `sysctl` — увидите только `sysctl`-объекты. Перепутать невозможно.

Поля

Поле	Пояснение
Тип проверки	Тот же, что у объекта и состояния
Идентификатор	Подставляется сам
Объект	Что проверяем — выбор из списка
Состояние	С чем сравниваем. Можно оставить пустым
Условие проверки	Сколько найденных объектов должны совпасть с состоянием
Условие существования	Сколько объектов вообще должно найтись
Комментарий	Попадёт в критерий на шаге 7 автоматически

Два условия, которые путают

Это разные вопросы, и отвечают на них по очереди:

5. **Условие существования** — сначала: нашли ли вообще то, что ищем?
6. **Условие проверки** — потом: среди найденного — сколько совпало с состоянием?

Почему это важно: если файл `rwquality.conf` вообще отсутствует, сравнивать нечего. Именно условие существования решает, считать ли это нарушением.

Значение условия существования	Смысл	Когда нужно
Существует хотя бы один	Найдено как минимум одно совпадение	Почти всегда. Значение по умолчанию
Не существует ни одного	Не должно найтись ничего	Проверка отсутствия: порт закрыт, пакет не установлен
Существуют все	Найдены все ожидаемые	Редко
Существует ровно один	Ни больше, ни меньше одного	Редко

Условие проверки в большинстве случаев остаётся Все: если параметр задан в файле дважды, правильными должны быть оба значения. Значение Хотя бы один опасно: одна верная строка скроет неверную.

Пример 1. Обычная проверка значения

Поле	Значение
Тип проверки	Содержимое текстового файла
Объект	oval:custom:obj:1 — Значение minlen
Состояние	oval:custom:ste:1 — Ожидаемое значение minlen
Условие проверки	Все
Условие существования	Существует хотя бы один

Пример 2. Проверка наличия — без состояния

Пакет auditd должен быть установлен. Версия не важна — сравнивать не с чем.

Поле	Значение
Тип проверки	Пакет DEB (dpkg)
Объект	oval:custom:obj:5 — Пакет auditd
Состояние	*(пусто)*
Условие существования	Существует хотя бы один

Пример 3. Проверка от обратного

Порт telnet не должен прослушиваться. Здесь нарушением считается сам факт находки.

Поле	Значение
Тип проверки	Прослушиваемые порты
Объект	Порт 23/tcp
Состояние	*(пусто)*
Условие существования	Не существует ни одного

Это единственный способ сформулировать требование «этого быть не должно».

Шаг 7. Определения — законченная проверка

До этого шага всё было техникой: пути, шаблоны, числа. Определение даёт проверке человеческое имя и описание — именно их увидит администратор в отчёте.

Поля

Поле	Пояснение
Идентификатор	Подставляется сам
Класс	Почти всегда Соответствие требованиям
Платформа (семейство ОС)	Windows или Linux. Выставляется автоматически
Заголовок	Краткое название проверки
Описание	Что именно проверяется и какой результат ожидается
Платформы	Конкретные системы, свободный текст
Критерии	Тесты, из которых складывается проверка

Семейство ОС и платформы — это разное

Два похожих поля, которые легко перепутать:

	Платформа (семейство)	Платформы
Что это	Закрытый список из двух вариантов	Свободный текст
Значения	Windows или Linux	Astra Linux, РЕД ОС 8
Кто заполняет	Конструктор по типу теста	Пользователь

Все дистрибутивы Linux — Astra Linux, РЕД ОС, ALT Linux, Ubuntu, Debian — относятся к одному семейству **Linux**. Конкретная система пишется в поле «Платформы» и нужна для отчётности.

Критерии: один тест или несколько

Критерий — это ссылка на тест. Чаще всего он один, но можно добавить ещё кнопкой **«+ Критерий»**. Если тестов несколько, проверка считается пройденной, когда выполнены **все** из них.

Когда это полезно: требование звучит как одно, а проверяется в нескольких местах. Например, «удалённый доступ по SSH настроен безопасно» — это сразу три параметра в `sshd_config`:

Критерий	Тест
1	PermitRootLogin равен no
2	PermitEmptyPasswords равен no
3	Protocol равен 2

Проверка пройдена только при выполнении всех трёх.

Как выбрать. Если в отчёте нужно видеть, какой именно параметр нарушен, делайте три отдельных определения. Если важен только общий итог — одно с тремя критериями.

Что конструктор делает сам

При выборе теста в критерии:

- комментарий критерия заполняется из комментария теста
- семейство ОС выставляется по типу проверки

Пример

Поле	Значение
Заголовок	Минимальная длина пароля
Описание	Проверяется, что значение параметра <code>minlen</code> больше или равно 14.
Платформа	Linux
Платформы	Astra Linux
Критерий	oval:custom:tst:1

Описание стоит писать так, чтобы оно было понятно без контекста: человек увидит его в отчёте среди сотни других строк.

Шаг 8. Правила — требование в документе

Здесь осуществляется переход из OVAL в XCCDF — из технического файла в документ требований. Правило — это пункт «так должно быть», а определение под ним — способ это проверить.

Шаг почти автоматический: выбираете определение — заголовок, описание и идентификатор требования заполняются сами. Остаётся проверить текст и поправить формулировку под язык ваших регламентов.

Поля

Поле	Пояснение
OVAL-файл	В каком файле искать определение
Определение OVAL	Сама проверка — выбор из списка
Идентификатор	Подставляется сам
Заголовок	Формулировка требования
Описание	Зачем требование нужно
Идентификатор требования	Обязателен. См. ниже
Выбрано по умолчанию	Оставьте снятым

Идентификатор требования — самое важное поле шага

Это ссылка на пункт внешнего документа, которому соответствует правило. Состоит из двух частей:

- **Система** — откуда требование: `custom_requirements`, ссылка на регламент, URL стандарта
- **Значение** — номер пункта: `REQ.1`, `5.4.1`, `ПУНКТ-12`

Без этого поля правило не будет распознано при импорте. Валидатор предупредит об этом на шаге 10.

Зачем это нужно: когда проверка найдёт нарушение, администратор должен понять, какой пункт какого документа не выполнен. Это то, что попадает в отчётность по соответствию.

Можно добавить несколько идентификаторов — когда одна и та же настройка требуется сразу несколькими документами:

Система	Значение
внутренний_регламент	ПБ-2024-14
отраслевой_стандарт	5.4.1

Галочка «Выбрано по умолчанию»

Она не включает правило в профиль — это делается на шаге 9. Оставляйте снятой: состав каждого профиля задаётся явно, и так понятнее.

Пример

Поле	Значение
OVAL-файл	checks-oval.xml
Определение OVAL	oval:custom:def:1 — Минимальная длина пароля
Идентификатор	xccdf_custom_rule_password_minlen
Заголовок	Минимальная длина пароля
Описание	Необходимо убедиться, что минимальная длина пароля — не менее 14 символов.
Идентификатор требования	custom_requirements / REQ.1

Чем правило отличается от определения

Заголовки часто совпадают, и возникает вопрос, зачем нужны оба элемента:

	Определение	Правило
Где живёт	OVAL-файл	XCCDF-файл
О чём говорит	Как проверить	Что требуется и по какому документу
Переиспользование	Одно на несколько правил	Входит в несколько профилей

Пример разделения: проверка «длина пароля ≥ 14 » одна, а требуют её и внутренний регламент, и отраслевой стандарт — два правила с разными идентификаторами требования ссылаются на одно определение.

Шаг 9. Профили — набор правил для запуска

Профиль отвечает на вопрос: какие именно правила проверять вместе, за один запуск. Именно профиль администратор выбирает в системе, назначая проверку на группу узлов.

Список правил с галочками встроен прямо в форму создания, и **по умолчанию отмечены все**. Если нужны все правила — просто нажмите «Создать».

Поля

Поле	Пояснение
Идентификатор	Подставляется сам
Заголовок	Название, видимое администратору при выборе проверки
Описание	Для каких узлов и зачем
Ссылка на источник	Необязательно: адрес регламента
Правила профиля	Галочки: кто входит в набор

Зачем несколько профилей

Один бенчмарк может содержать несколько профилей с разным составом — правила при этом общие, дублировать ничего не нужно:

Профиль	Состав	Для кого
Базовый	Минимальный набор	Рядовые серверы
Усиленный	Базовый плюс жёсткие требования	Сегмент обработки данных
Аудит	Только проверки журналирования	Разбор инцидентов

Так же удобно разворачивать требования постепенно: сначала «Базовый» на всём парке, через квартал — «Усиленный».

Изменение состава позже

В таблице профилей есть кнопка **«Состав»** — тот же список галочек. Добавили новое правило на шаге 8 — вернитесь сюда и отметьте его в нужных профилях: автоматически оно туда не попадёт.

В столбце «Правил в составе» видно соотношение вида 4 / 5 — сколько правил вошло из всех имеющихся. Быстрый способ заметить забытое правило.

Пример

Поле	Значение
Идентификатор	xccdf_custom_profile_astra
Заголовок	Базовый профиль Astra Linux
Описание	Базовый набор проверок конфигурации безопасности для серверов подразделения.
Правила	Отмечены все пять

Шаг 10. Проверка и экспорт

Заключительный шаг: убедиться, что всё связано правильно, и забрать архив. Кнопка **«Проверить проект»** запускает валидатор.

Ожидаемый результат:

Проект соответствует требованиям, ошибок и замечаний не найдено.

Что проверяется

Валидатор смотрит на связность и формат, а не на смысл проверок:

- обязательные поля и уникальность идентификаторов
- формат идентификаторов по схеме OVAL
- цепочку ссылок: профиль → правило → определение → тест → объект и состояние
- совпадение типов проверки у связанных элементов
- наличие идентификатора требования у каждого правила

Что валидатор **не** проверяет: правильно ли написан путь к файлу, существует ли пакет с таким именем, верно ли составлено регулярное выражение. Это выяснится только при запуске на узле.

Ошибки и замечания

Уровень	Что значит
Ошибка	Архив не будет принят продуктом. Исправлять обязательно
Замечание	Формально допустимо, но скорее всего не соответствует первоначальной цели.

Типичное замечание — правило без идентификатора требования. Формально такой архив соберётся, но система не сможет сопоставить правило со своей базой требований.

Структура архива

Кнопка **«Экспортировать ZIP»** собирает:

profile.zip	
benchmark-xccdf.xml	требования, правила, профили
checks-oval.xml	техническая логика проверок

Оба файла лежат в корне архива, без вложенных папок — это требование формата. Распаковывать и переупаковывать его не нужно.

Повторный экспорт и доработка

Проект остаётся в конструкторе и после экспорта — можно вернуться, поправить и выгрузить снова. Если архив собирался не здесь, его можно загрузить через **«Импорт ZIP...»** в левой панели и доработать.

Три сквозных примера

Три проверки от простой к составной — целиком, от переменной до правила.

Пример 1. Пакет аудита установлен

Самый короткий путь: без переменной и без состояния. Проверяется сам факт наличия.

Шаг	Что создаём
3. Переменные	—
4. Объекты	Тип Пакет <code>DEB (dpkg)</code> , имя пакета <code>auditd</code>
5. Состояния	—
6. Тесты	Объект — пакет <code>auditd</code> , состояние пустое, условие существования <code>Существует хотя бы один</code>
7. Определения	Наличие подсистемы аудита, платформа <code>Linux</code>
8. Правила	Ссылка на определение, <code>ident REQ. 5</code>

Три элемента вместо шести — потому что сравнивать не с чем.

Пример 2. Переадресация пакетов отключена

Полная цепочка с переменной и числовым сравнением.

Шаг	Что создаём
3. Переменные	Тип Целое число, значение <code>0</code>
4. Объекты	Тип Параметр <code>sysctl</code> , имя <code>net.ipv4.ip_forward</code>
5. Состояния	Поле <code>value</code> : из переменной, операция <code>Равно</code> , тип Целое число
6. Тесты	Объект + состояние, условия по умолчанию
7. Определения	Отключение переадресации IP-пакетов
8. Правила	<code>ident REQ. 3</code>

Главная тонкость примера — тип данных. У `sysctl` по умолчанию стоит Строка; если его не переключить, сравнение пойдёт по алфавиту.

Пример 3. Минимальная длина пароля

Самый сложный случай: значение извлекается из текстового файла.

Шаг	Что создаём
3. Переменные	Тип Строка, значение <code>^(1[4-9]</code>
4. Объекты	Путь <code>/etc/security</code> , файл <code>pwquality.conf</code> , шаблон <code>^[[:space:]]*minlen[[:space:]]*=[[:space:]]*([0-9]+)</code>
5. Состояния	<code>subexpression</code> : из переменной, операция <code>Соответствует регулярному выражению</code> , ТИП Строка
6. Тесты	Объект + состояние
7. Определения	Минимальная длина пароля

Шаг	Что создаём
8. Правила	ident REQ.1

Почему не «больше или равно 14»: значение пришло строкой из файла, а строки сравниваются по алфавиту. Поэтому диапазон «14 и больше» записывается шаблоном.

Сравнение трёх подходов

	Проверка наличия	Значение от системы	Значение из файла
Переменная	Не нужна	Полезна	Полезна
Состояние	Не нужно	Нужно	Нужно
Операция	—	Сравнение чисел	Шаблон
Тип данных	—	Целое число	Строка

Механики интерфейса

Мастер и конструктор

Два режима переключаются кнопками над шагами. Данные у них общие — это один и тот же проект, показанный по-разному.

	Мастер	Конструктор
Порядок	Десять шагов подряд	Вкладки в любом порядке
Подсказки	На каждом шаге	Нет
Кому удобнее	Первый профиль	Правка готового

Переключаться можно в любой момент, ничего не теряется. Выбор запоминается в браузере.

Язык и тема

Две кнопки в левом верхнем углу:

- **RU / EN** — язык интерфейса. По умолчанию русский
-  /  — светлая или тёмная тема. По умолчанию светлая

Язык меняет только интерфейс. Ваши тексты — заголовки правил, описания — остаются как введены. Термины стандарта (definition, ident, check-content-ref) не переводятся ни на одном языке: они совпадают с именами элементов в XML.

Редактирование и удаление

У определений, правил и профилей в таблицах есть кнопка **«Изменить»**. Объекты, состояния, тесты и переменные правке не подлежат — их удаляют и создают заново.

Удалять нужно сверху вниз, в обратном порядке к созданию: сначала правило, потом определение, потом тест. Иначе останутся ссылки в никуда, и валидатор сообщит об ошибке.

Импорт готового архива

Кнопка **«Импорт ZIP...»** в левой панели загружает архив как новый проект. Полезно в трёх случаях:

7. Доработать профиль, собранный раньше или коллегой
8. Взять готовый профиль за основу для своего
9. Посмотреть, как устроены чужие проверки

Хранение данных

Проекты лежат на сервере в /opt/compliance-profile-builder/data/projects/ — по папке на проект, внутри обычные XML-файлы. Сохранение происходит сразу при нажатии «Создать» или «Сохранить» — отдельной кнопки сохранения проекта нет.

Частые ошибки

Валидатор нашёл проблему

Сообщение	Причина	Что сделать
Правило не содержит ident	Не заполнен идентификатор требования	Открыть правило, заполнить систему и значение
Ссылка на несуществующий элемент	Удалён элемент, на который ссылаются	Создать заново или удалить ссылающегося
Формат идентификатора неверен	Идентификатор правила вручную	Нужен вид <code>oval: <пространство> : <тип> : <число></code>
Профиль не содержит правил	Сняты все галочки	Кнопка «Состав», отметить правила
Неизвестные поля для этого типа	Опечатка в имени поля	Допустимые имена перечислены в тексте ошибки

В списке нечего выбрать

Ситуация	Причина
Пуст список объектов в тесте	У объекта другой тип проверки — список фильтруется по типу
Пуст список состояний	То же самое, либо состояние не создавалось
Пуст список определений в правиле	Выбран другой OVAL-файл
Вместо списка — плашка с кнопкой	Нет элемента предыдущего шага — нажмите кнопку перехода

Профиль загрузился, но результат неожиданный

Эти ошибки валидатор не ловит — формально всё верно:

Симптом	Вероятная причина
Проверка всегда проходит, даже где не должна	Состояние не привязано к тесту — проверяется только наличие
Числовое сравнение врёт	Тип данных Строка вместо Целое число
Сравнивается вся строка файла	В шаблоне нет скобок () вокруг нужной части
Нарушение не видно, хотя оно есть	Условие проверки Хотя бы один вместо Все
Проверка не применилась к узлу	Неверное семейство ОС в определении

Как проверить себя

Запустите профиль на узле, где заранее сломана одна настройка. Если эта проверка прошла успешно — она не работает. Профиль, где всё зелёное с первого раза, чаще всего не проверяет ничего.

Словарь терминов

Слева — как называется в интерфейсе, справа — как записывается в XML. Второе пригодится, если понадобится заглянуть в файлы или обсудить проблему с поддержкой.

В интерфейсе	В XML	Что это
Бенчмарк	Benchmark	Весь документ требований
Профиль	Profile	Набор правил для одного запуска
Правило	Rule	Одно требование безопасности
Идентификатор требования	ident	Ссылка на пункт внешнего документа
Состав профиля	select	Отметка, что правило входит в профиль
Файл и определение проверки	check-content-ref	Ссылка из правила в OVAL-файл
Определение	definition	Законченная проверка с названием
Критерий	criterion	Ссылка из определения на тест
Тест	test	Связка объекта и состояния
Объект	object	Где искать
Состояние	state	С чем сравнить
Переменная	constant_variable	Эталонное значение
Из переменной	var_ref	Ссылка состояния на переменную
Условие проверки	check	Сколько найденных должны совпасть
Условие существования	check_existence	Сколько должно найтись вообще
Платформа (семейство ОС)	affected/@family	Windows или Linux
Платформы	platform	Конкретные системы, свободный текст

Два формата

	XCCDF	OVAL
О чём	Что требуется	Как это проверить
Файл	benchmark-xccdf.xml	checks-oval.xml
Главные элементы	Профиль, правило	Определение, тест, объект, состояние
Кто читает	Человек в отчёте	Сканер на узле

Структура идентификатора OVAL

Все идентификаторы имеют вид `oval:<пространство>:<тип>:<число>`, например `oval:custom:obj:3:`

Часть	Значение
def	Определение
tst	Тест
obj	Объект
ste	Состояние
var	Переменная

Зная это, по любому идентификатору из сообщения об ошибке сразу понятно, на каком шаге искать проблему.